
Preventative Health Customization of local LLMs through PEFT and RAG

Gabriel Pacheco

Bradley Department of Electrical
& Computer Engineering
Virginia Tech

Blacksburg, Virginia, United States of America

Abstract

This study presents the development of a pipeline for customizing large language models (LLMs) through the combined use of Parameter-Efficient Fine-Tuning (PEFT) and Retrieval-Augmented Generation (RAG). The work involves constructing a Supervised Fine-Tuning (SFT) dataset to fine-tune a Qwen2.5-7B model, enabling it to generate general preventative healthcare guidance without producing specialized medical advice. Low-Rank Adaptation (LoRA) is employed to achieve lightweight, memory-efficient fine-tuning. Documents are chunked and embedded so that relevant sections can be retrieved and merged with user prompts at inference time, enabling contextualized responses. This integrated approach explores how PEFT and RAG can be combined to produce a more adaptable and domain-aware LLM.

1 INTRODUCTION AND MOTIVATION

With the rapid release of AI models into the market, it has become increasingly common for companies to look for ways to incorporate these tools into their workflows. However, according to a study from MIT, around 95% of AI initiatives in companies fail because organizations adopt AI merely for the sake of appearing innovative, rather than addressing real operational needs, integrating these tools into their existing systems, and using company data [1].

Motivated by this insight, the goal of the project is to develop a framework that uses Parameter-Efficient Finetuning (PEFT) and Retrieval-Augmented Generation (RAG) to tailor an LLM to a specific context. Such a framework has the potential to turn Large Language Models (LLMs) into specialized tools for context-aware support.

The aim is to design it in a way that does not require heavy training or large memory resources, allowing small institutions that need context-aware solutions to implement it effectively.

RQ: Can lightweight fine-tuning techniques (SFT and LoRA) combined with RAG create a cost-efficient LLM prototype that accurately reflects company-specific knowledge?

2 BACKGROUND

2.1 LARGE LANGUAGE MODELS

LLMs are a type of artificial intelligence program designed to recognize and generate text. These models are trained on large amounts of data and use neural networks called transformers. They are built to understand relationships between words and how characters, words, and sentences work together. As explained by Cloudflare, they can be further trained or fine-tuned for specific tasks the developer wants them to perform, such as interpreting questions or generating responses in a particular way.

These models have a wide range of applications, including chatbots, sentiment analysis, coding assistance, customer service, and online search. Real-world examples include ChatGPT, Bard, Llama, and GitHub Copilot [2].

2.2 SUPERVISED FINE-TUNING

To train LLMs towards specific tasks there are multiple methods. One of them is Supervised Fine-Tuning (SFT), which according to HuggingFace is a critical process for adapting pre-trained languages models to specific tasks using a task-oriented dataset with labeled examples. This process is performed after the complete training of the base model [3].

2.3 LOW RANK ADAPTATION (LoRA)

Low-Rank Adaptation (LoRA) is a fine-tuning technique that enhances large language models by introducing low-rank matrices into selected layers of the network. Instead of retraining the entire model, which is computationally expensive and memory-intensive, LoRA updates only these additional matrices. This approach enables the model to learn task-specific patterns while preserving the original pre-trained weights.

The output of this process is an adapter module that sits on top of the base model. Because the core parameters remain frozen, training becomes significantly faster and more efficient. The primary advantage of LoRA is the substantial reduction in memory and computational requirements, making it feasible to fine-tune large models in resource-constrained environments [3].

3 RELATED WORK

In the academic space, several studies have explored techniques for adapting large language models to specific domains. Shukla et al. present a comparison between RAG and PEFT as two distinct approaches for LLM customization, rather than examining their combined use. Their work highlights that fine-tuning excels at capturing domain-specific nuances with high accuracy, while RAG offers unparalleled adaptability by incorporating external knowledge at runtime [4]. Integrating both methods, as done in this project, may provide complementary benefits greater than either technique alone.

Another relevant study by Sung et al. introduces a pipeline that leverages both RAG and fine-tuning, but with a different objective. In their approach, retrieved documents are used to generate synthetic instruction datasets, which are then employed for fine-tuning [5]. In contrast, the present project incorporates RAG only at inference time, using retrieved context directly for real-time reasoning rather than as part of dataset construction.

Industry applications also reflect similar ideas. For example, Databricks provides enterprise-level AI agents trained on proprietary business data, demonstrating a real-world implementation of context-aware, domain-specific LLMs [6]. While aligned in concept, such solutions operate at a

significantly larger scale, supported by extensive infrastructure and resources not available in smaller or low-resource environments.

4 METHODOLOGY

This project was developed with the objective of adapting a large language model (LLM) to prevention healthcare advising context through a lightweight pipeline suitable for machines with limited computational resources. The methodology is organized into three main stages:

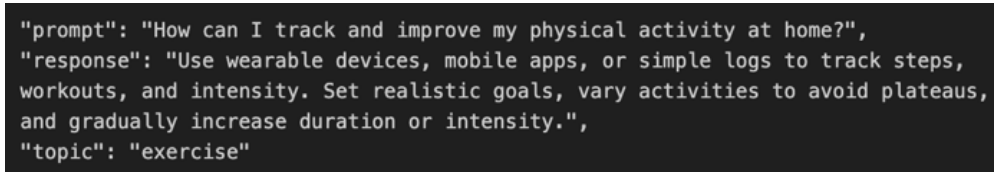
1) Development of the task-specific SFT dataset: A supervised fine-tuning (SFT) dataset was constructed to reflect the task at hand. This dataset contains curated examples designed to guide the model toward the desired behavior and contextual understanding.

2) Fine-tuning of the LLM through LoRA using SFT dataset: The selected LLM was fine-tuned using the task-specific dataset, leveraging parameter-efficient techniques to minimize memory usage and avoid retraining the full model. This stage adapts the model’s behavior toward the target task while preserving its foundational knowledge.

3) Retrieval-Augmented Generation (RAG) Integration: Contextual documents relevant to the domain incorporated through a RAG pipeline. During inference, retrieved documents are merged with the user prompt to provide the model with up-to-date, context-aware information, enhancing accuracy and relevance.

4.1 DATASET CONSTRUCTION

The process began with the development of a supervised fine-tuning (SFT) dataset tailored to the context of a preventive-health advisory chatbot. The goal was to generate examples that provided general, safe, and non-medical guidance while modeling the response style the system should follow. To initiate this, ChatGPT was used to create an initial set of 12 prompt–response–topic pairs simulating scenarios in which a user requests general health or wellness advice. Each pair demonstrated the expected answer format and conversational tone. An example pair received is shown in Figure 1.



```
"prompt": "How can I track and improve my physical activity at home?",  
"response": "Use wearable devices, mobile apps, or simple logs to track steps,  
workouts, and intensity. Set realistic goals, vary activities to avoid plateaus,  
and gradually increase duration or intensity.",  
"topic": "exercise"
```

Figure 1: **SFT dataset instance pair example.** The screenshot shows an example of an instance pair created by ChatGPT which provides a prompt, a response, and a topic in relation to general advice. This is one of the many examples provided by the model which is further on expanded to create an SFT dataset.

All initial examples were manually reviewed to ensure that the chatbot’s behavior aligned with the project’s requirements: that no medical or diagnostic advice was included and that the content remained within the boundaries of general preventive guidance.

After validating this initial set, a script was implemented to automate further dataset generation. Using LangChain and the Qwen2.5-7B model, the script prompted the model to generate additional examples following the same structure and context, framed as a professional preventive-health assistant. Although the target was 50 examples per execution, the model did not always produce the full set, however, it still consistently generated a substantial number of high-quality instances. Each new example was automatically appended to a JSON file containing all prompts, responses, and associated topics.

This stage was successful, as the dataset was expanded and maintained consistency with the intended behavioral guidelines. The final SFT dataset consisted of 473 instances.

4.2 MODEL FINE-TUNING

Once the SFT dataset was prepared, the next step involved adapting the base LLM using a parameter-efficient fine-tuning approach with LoRA. The Qwen2.5-7B model was first loaded as a 16-bit quantized model to reduce memory consumption. LoRA modules were then added to the model, targeting the query and value projection layers. The LoRA configuration was set with a rank of 16, an alpha of 32, and a dropout rate of 0.05, allowing the model to learn task-specific patterns without modifying its original weights.

The SFT dataset, stored in JSON format, was converted into a Hugging Face dataset to be compatible with the tokenizer and Trainer API. Each example was tokenized by concatenating the topic, instruction, and response into a single string, which was then converted into tensors suitable for training. A data collator standardized these tensors to ensure consistency across batches.

Fine-tuning was performed over three epochs with a batch size of four and a learning rate of 2×10^{-4} . During training, only the LoRA adapter parameters were updated, preserving the base model's pre-trained knowledge while guiding it to generate responses consistent with the tone and style of a preventive-health advisory chatbot.

The resulting model retains the foundational knowledge of Qwen2.5-7B, augmented with a set of LoRA adapters that encode domain-specific behavior. This approach allows the model to provide context-aware, safe, and non-medical guidance.

4.3 CONTEXT INTEGRATION VIA RAG

The final stage of the system involved integrating Retrieval-Augmented Generation (RAG) to provide the model with domain-relevant context during inference. In this project, the goal was to supply general preventive-health information without generating specific or diagnostic medical advice. RAG enables this by retrieving relevant passages from external documents and incorporating them into the prompt before the model generates a response.

To prepare the contextual knowledge base, the selected document "*Health2030*", a long-term national health strategy published by the Swiss government, was processed through a series of steps [7]. The document was first loaded and divided into smaller, semantically coherent chunks using a Recursive Character Text Splitter. These chunks were then converted into vector embeddings using the BAAI/bge-small-en-v1.5 embedding model. The embeddings were stored in a FAISS vector database, which allows efficient similarity search based on the user's query.

At inference time, the system retrieves the most relevant text chunks by comparing the query to the stored embeddings. The retrieved passages are combined into a structured prompt containing (1) the contextual excerpts, (2) the user's question, and (3) an instruction ensuring that the output remains general, safe, and aligned with preventive-health guidelines.

The generation step uses the Qwen2.5-7B model augmented with the previously trained LoRA adapters. When the prompt is passed to the model, a low temperature (0.3) is used to encourage controlled, consistent behavior. A complete scope of the model can be seen in Figure 2 down below.

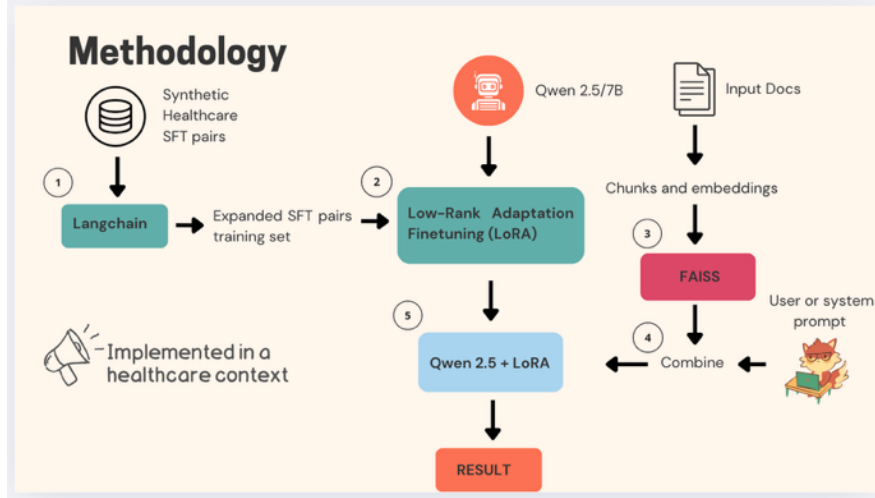


Figure 2: **System Workflow.** This image shows the flow behind the pipeline. We can see that it first looks to expand the examples provided by ChatGPT into an SFT dataset. Further upon, LoRA is used to finetune the Qwen2.5/7B model using the dataset previously created. Finally, this finetuned model is called by a prompt that is combined at inference time with context provided by a RAG system.

5 RESULTS

The pipeline was executed in two environments to assess results and model-loading behavior: (1) a 13-inch MacBook Air with an Apple M3 chip and 16 GB of memory, and (2) Virginia Tech’s ARC computing cluster when run with a GPU and 32 GB of memory.

Across both environments, the overall pipeline ran successfully up to the final inference stage, where the system retrieves contextual passages from the FAISS index and feeds them into the model. However, significant differences emerged in how each system handled model loading and inference.

5.1 MacBook Air (M3, 16 GB RAM)

On the M3 machine, loading the model in a 16-bit format exceeded available memory and caused repeated crashes. The model could not be loaded quantized to 8-bit and run through CUDA due to them not supporting iOS. Experiments that attempted to offload part of the model to storage (to reduce RAM pressure) resulted in extremely slow generation times. The latency made the pipeline impractical for any real-world scenario, leading to the conclusion that the pipeline, as designed, is not sustainable on a non-GPU system with limited memory. A new loading strategy, such as more aggressive quantization or a smaller model would be required for CPU-only environments.

5.2 ARC GPU Cluster (32 GB RAM)

In contrast, the ARC system was able to load and run the 16-bit quantized model without issue. Retrieval from the vector database and prompt construction happened efficiently, and inference completed without memory errors. This confirmed that the architecture works as intended when sufficient RAM is available.

However, qualitative evaluation of the model outputs highlighted an important limitation: although the model successfully integrated retrieved passages into its responses, the generated answers did not clearly align with the intended “preventative-health guidance” goal. Instead, the model produced

high-level summaries of the source document and general commentary on the health topics it contained. The responses did not present structured or actionable prevention advice.

This indicates that, while retrieval was functioning mechanically, it was not consistently pulling the most relevant sections or framing them in the desired way. The model appeared to receive context, but not the *right type* of context to guide the output toward prevention-focused recommendations. This suggests that the retrieval pipeline requires refinement, potentially through improved chunking and better embeddings.

6 CONCLUSION

This project demonstrated the feasibility of integrating supervised fine-tuning, and retrieval augmented generation into a unified platform to provide customization to LLMs for diverse use cases, if so desired.

However, the evaluation also revealed two central limitations. First, hardware constraints strongly influenced model performance. On a consumer-grade MacBook Air, even 16-bit quantized models exceeded available memory, and alternative loading strategies led to unusable latency. This indicates that running models of this scale in CPU-only environments is not currently practical without further optimization, such as smaller base models or more aggressive quantization.

Second, although the ARC GPU system handled the model efficiently, the quality of the generated responses showed that retrieval alone is not enough to ensure task-aligned outputs. The model integrated contextual passages but produced broad summaries rather than focused preventative-health recommendations. This suggests that improvements are needed in how context is selected, how prompts guide the model, or how the model itself is fine-tuned for specificity.

7 FUTURE WORK

Building on the limitations identified in this project, three primary areas should guide future work:

- 1) Future efforts should therefore focus on techniques that reduce the computational load of the model. Potential directions include exploring more aggressive quantization strategies, adopting smaller foundation models (e.g., 3B parameters instead of the 7B model used here), or applying model distillation to create lighter variants tailored to the task. These approaches would move the system closer to a deployment scenario that is affordable and practical for small organizations or low-resource environments.
- 2) The second area of improvement concerns the RAG component. Although the current pipeline successfully retrieves and provides context, the results show that the retrieved passages are often broad chunks of the documents related to the topics on the prompt. To address this, future work should focus on retrieval methods that increase relevance while reducing unnecessary token load. A promising direction is hierarchical, section-level chunking, which allows the system to select only the most relevant subsections of a document and produce smaller, more focused context windows.
- 3) The final area of improvement is to develop a quantitative evaluation method for the pipeline. This would make it possible to measure its accuracy, compare its performance against baseline models, and establish a clear benchmark for future iterations.

REFERENCES

- [1] Challapally, A., Pease, C., Raskar, R., & Chari, P., “The GenAI Divide : State of AI in Business 2025”, MIT NANDA, July 2025.
- [2] Cloudflare, “What is a large language model (LLM)?”, *Cloudflare.com*.
<https://www.cloudflare.com/learning/ai/what-is-large-language-model/>

- [3] HuggingFace, “Supervised Fine-Tuning”, huggingface.co. <https://huggingface.co/learn/llm-course/chapter11/1>
- [4] V. Shukla and G. G. Parker, "Building Custom Large Language Models for Industries: A Comparative Analysis of Fine-Tuning and Retrieval-Augmented Generation Techniques", *2024 International Conference of Adisutjipto on Aerospace Electrical Engineering and Informatics (ICAAEEI)*, Yogyakarta, Indonesia, 2024, pp. 1-6, doi: 10.1109/ICAAEEI63658.2024.10899129.
- [5] C. Sung, Y. Lee, and Y. Tsai, “A new pipeline for generating instruction dataset via RAG and self fine-tuning”, 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), doi: [arXiv:2408.05911](https://arxiv.org/abs/2408.05911)
- [6] “Data Lakehouse Architecture and AI Company” , *Databricks*. <https://www.databricks.com/>
- [7] “Health2030,” *Admin.ch*, 2024. <https://www.bag.admin.ch/en/health2030>